

一、环境准备

pc 系统: Ubuntu 16.04 LTS

```
sudo apt-get install git
```

二、下载代码

我们需要下载 lvgl v8.2 版本的三个代码包如下（代码在压缩包）

1、lvgl 核心绘图代码

```
git clone https://github.com/lvgl/lvgl.git 切到 tag v8.2
```

2、lv_drivers 设备相关中间驱动层

```
git clone https://github.com/lvgl/lv_drivers.git 切到 tag v8.2
```

3、lv_port_linux_frame_buffer 即用工程 linux ARM

```
git clone https://github.com/lvgl/lv\_port\_linux\_frame\_buffer
```

注意：直接使用 git 下载的代码在 master 分支，我们需要把代码切换更新在 v8.2 版本。

方法：

1. Cd 进入各自对应目录
2. Git status 查看目前分支（应该是 master）
3. Git branch -a 查看代码的分支
4. Git checkout + 分支版本号 切换到 8.2 版本
5. Git status 查看目前版本号

三、搭建自己的即用代码工程

1. 创建一个自己的目录，比如：my_lvgl
2. 将 lv_port_linux_frame_buffer 即用工程拷贝到自己的目录下；
3. lv_port_linux_frame_buffer（以下简称即用工程）中的一级子目录 lvgl、lv_drivers 为空文件夹，需要我们手动去完成代码移动所以需要：

(1)将 lvgl 复制并替换到 lv_port_linux_frame_buffer 下的 lvgl

(2)将 lv_drivers 复制并替换到 lv_port_linux_frame_buffer 下的 lv_drivers

此时其实可以直接 make 编译了：

编译出来的 demo 可以直接在真终端中显示。

其中有个报错：

unable to open evdev interface:: No such file or directory

跟踪代码：

发现是在打开 event10 时报错，我们/dev/input 下没有这个设备文件。这是个输入有关，我们先不管。

四、将工程向开发板移植

1. 修改 Makefile

1) 先修改编译器

2) 编译时发现报一些编译选项的错误：

```
arm-linux-gnueabihf-gcc: error: unrecognized
command line option ‘-Wshift-negative-value’
```

```
Makefile:35: recipe for target
‘/home/linux/lvgl/my_lvgl/lv_port_linux_frame_buffe
r/lvgl/demos/music/lv_demo_music.o’ failed
```

```
make: ***
[/home/linux/lvgl/my_lvgl/lv_port_linux_frame_buffe
r/lvgl/demos/music/lv_demo_music.o] Error 1
```

这是因为我们用的编译器版本没有 gcc 版本高, 目前编译器不支持这些选项。可将选项都注掉。

```
#
# Makefile
#
CC = arm-none-linux-gnueabi-gcc
LVGL_DIR_NAME ?= lvgl
LVGL_DIR ?= ${shell pwd}
CFLAGS ?= -O3 -g0 -I$(LVGL_DIR)/ -std=c99 #-Wall -Wshadow -Wundef -Wmissing-prototypes -Wno-discarded-qualifiers -Wall
-Wextra -Wno-unused-function -Wno-error=strict-prototypes -Wpointer-arith -fno-strict-aliasing -Wno-error=cpp
-Wuninitialized -Wmaybe-uninitialized -Wno-unused-parameter -Wno-missing-field-initializers -Wtype-limits
-Wsizeof-pointer-memaccess -Wno-format-nonliteral -Wno-cast-qual -Wunreachable-code -Wno-switch-default -Wreturn-type
-Wmultichar -Wformat-security -Wno-ignored-qualifiers -Wno-error=pedantic -Wno-sign-compare
-Wno-error=missing-prototypes -Wdouble-promotion -Wclobbered -Wdeprecated -Wempty-body -Wtype-limits
-Wshift-negative-value -Wstack-usage=2048 -Wno-unused-value -Wno-unused-parameter -Wno-missing-field-initializers
-Wuninitialized -Wmaybe-uninitialized -Wall -Wextra -Wno-unused-parameter -Wno-missing-field-initializers -Wtype-limits
-Wsizeof-pointer-memaccess -Wno-format-nonliteral -Wpointer-arith -Wno-cast-qual -Wmissing-prototypes
-Wunreachable-code -Wno-switch-default -Wreturn-type -Wmultichar -Wno-discarded-qualifiers -Wformat-security
-Wno-ignored-qualifiers -Wno-sign-compare
LDFLAGS ?= -lm
BIN = demo
```

3. 修改代码

在 main.c 的 main 函数内部修改分辨率的配置

```
/*Initialize and register a display driver*/  
  
static lv_disp_drv_t disp_drv;  
  
lv_disp_drv_init(&disp_drv);  
  
disp_drv.draw_buf = &disp_buf;  
  
disp_drv.flush_cb = fbdev_flush;  
  
disp_drv.hor_res = 800;  
disp_drv.ver_res = 480;  
  
lv_disp_drv_register(&disp_drv);
```

在 lv_conf.h 中修改颜色位深的配置

```
/*=====
COLOR SETTINGS
=====*/  
  
/*Color depth: 1 (1 byte per pixel), 8 (RGB332), 16 (RGB565), 32 (ARGB8888)*/  
#define LV_COLOR_DEPTH 16  
  
/*Swap the 2 bytes of RGB565 color. Useful if the display has an 8-bit interface (e.g. SPI)*/  
#define LV_COLOR_16_SWAP 0  
  
/*Enable more complex drawing routines to manage screens transparency.  
*Can be used if the UI is above another layer, e.g. an OSD menu or video player.  
*Requires `LV_COLOR_DEPTH = 32` colors and the screen's `bg_opa` should be set to non LV_OPA_COVER value*/  
#define LV_COLOR_SCREEN_TRANSP 0
```

在 lv_drv_conf.h 中修改触摸屏节点路径

可以通过以下命令查看系统的输入设备：

Cat /proc/bus/input/devices

```

root@ATK-IMX6U:/mnt# cat /proc/bus/input/devices
I: Bus=0019 Vendor=0000 Product=0000 Version=0000
N: Name="20cc000.snvs:snvs-powerkey"
P: Phys=snvs-pwrkey/input0
S: Sysfs=/devices/platform/soc/2000000.aips-bus/20cc000.snvs/20cc000.snvs:snvs-powerkey/input/input0
U: Uniq=
H: Handlers=kbd event0
B: PROP=0
B: EV=3
B: KEY=100000 0 0 0

I: Bus=0018 Vendor=dead Product=beef Version=28bb
N: Name="goodix-ts"
P: Phys=input/ts
S: Sysfs=/devices/virtual/input/input2
U: Uniq=
H: Handlers=event1
B: PROP=3
B: EV=b
B: KEY=e520 0 0 0 0 0 0 0 0 0
B: ABS=2658000 0

I: Bus=0019 Vendor=0001 Product=0001 Version=0100
N: Name="gpio_keys@0"
P: Phys=gpio-keys/input0
S: Sysfs=/devices/platform/gpio_keys@0/input/input3
U: Uniq=
H: Handlers=kbd event2
B: PROP=0
B: EV=100003
B: KEY=40000 0 0 0

```

设备名称 触摸屏

设备节点

```

/*-----
 * Mouse or touchpad as evdev interface (for Linux based systems)
 *-----*/

#ifndef USE_EVDEV
# define USE_EVDEV 1
#endif

#ifndef USE_BSD_EVDEV
# define USE_BSD_EVDEV 0
#endif

#if USE_EVDEV || USE_BSD_EVDEV
# define EVDEV_NAME "/dev/input/event0" /*You can use the "evtest" Linux tool to get the list of devices and
test them*/

# define EVDEV_SWAP_AXES 0 /*Swap the x and y axes of the touchscreen*/
# define EVDEV_CALIBRATE 0 /*Scale and offset the touchscreen coordinates by using maximum and
minimum values for each axis*/

# if EVDEV_CALIBRATE
# define EVDEV_HOR_MIN 0 /*to invert axis swap EVDEV_XXX_MIN by EVDEV_XXX_MAX*/
# define EVDEV_HOR_MAX 4096 /*"evtest" Linux tool can help to get the correct calibraion values>*/
# define EVDEV_VER_MIN 0
# define EVDEV_VER_MAX 4096
# endif /*EVDEV_CALIBRATE*/
#endif /*USE_EVDEV*/

```

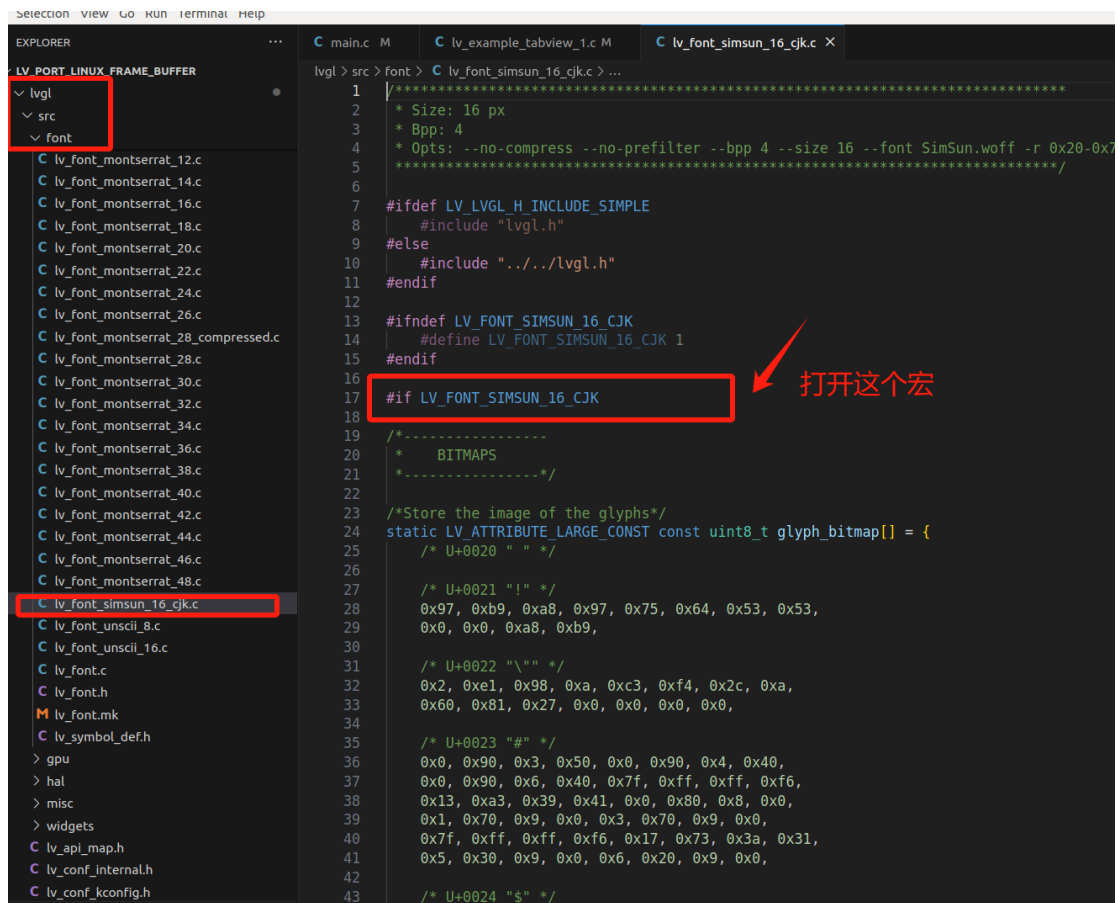
四、编译工程

执行 make 命令， 将可执行文件拷贝到挂载目录，执行测试结果。

五、相关错误问题

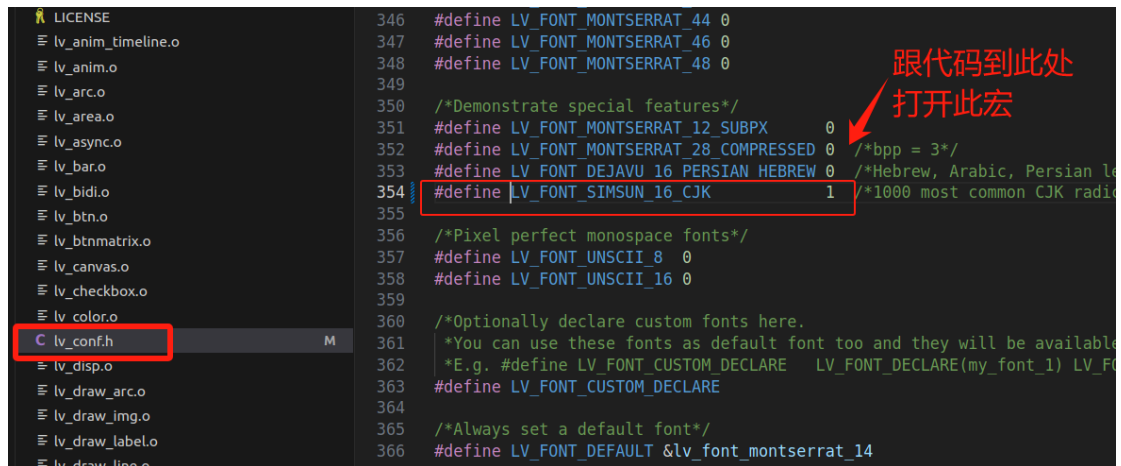
六、使用 LVGL 提供的字库显示中文

LVGL 提供的字库大概有一千左右个汉字，常用的基本覆盖。



The screenshot shows a code editor with three panes. The left pane is the 'EXPLORER' view showing a file tree for 'LV_PORT_LINUX_FRAME_BUFFER'. The middle pane shows the 'main.c' file with a comment 'lvgl > src > font > C lv_font_simsun_16_cjk.c > ...'. The right pane shows the 'lv_font_simsun_16_cjk.c' file. In the file tree, 'lvgl' is expanded, and 'font' is selected. In the code editor, the 'lv_font_simsun_16_cjk.c' file is selected, and the line '#if LV_FONT_SIMSUM_16_CJK' is highlighted with a red box. A red arrow points to this line with the text '打开这个宏' (Open this macro).

```
1  |*****|
2  |* Size: 16 px|
3  |* Bpp: 4|
4  |* Opts: --no-compress --no-prefilter --bpp 4 --size 16 --font SimSun.woff -r 0x20-0x7|
5  |*****|
6  |
7  |#ifdef LV_LVGL_H_INCLUDE_SIMPLE|
8  |    #include "lvgl.h"|
9  |#else|
10 |    #include "../lvgl.h"|
11 |#endif|
12 |
13 |#ifndef LV_FONT_SIMSUM_16_CJK|
14 |    #define LV_FONT_SIMSUM_16_CJK 1|
15 |#endif|
16 |
17 |#if LV_FONT_SIMSUM_16_CJK|
18 |
19 |/*-----|
20 |* BITMAPS|
21 |*-----*/|
22 |
23 |/*Store the image of the glyphs*/|
24 |static LV_ATTRIBUTE_LARGE_CONST const uint8_t glyph_bitmap[] = {|
25 |    /* U+0020 " " */|
26 |
27 |    /* U+0021 "!" */|
28 |    0x97, 0xb9, 0xa8, 0x97, 0x75, 0x64, 0x53, 0x53,|
29 |    0x0, 0x0, 0xa8, 0xb9,|
30 |
31 |    /* U+0022 "\" */|
32 |    0x2, 0xe1, 0x98, 0xa, 0xc3, 0xf4, 0x2c, 0xa,|
33 |    0x60, 0x81, 0x27, 0x0, 0x0, 0x0, 0x0,|
34 |
35 |    /* U+0023 "#" */|
36 |    0x0, 0x90, 0x3, 0x50, 0x0, 0x90, 0x4, 0x40,|
37 |    0x0, 0x90, 0x6, 0x40, 0x7f, 0xff, 0xff, 0xf6,|
38 |    0x13, 0xa3, 0x39, 0x41, 0x0, 0x80, 0x8, 0x0,|
39 |    0x1, 0x70, 0x9, 0x0, 0x3, 0x70, 0x9, 0x0,|
40 |    0x7f, 0xff, 0xff, 0xf6, 0x17, 0x73, 0x3a, 0x31,|
41 |    0x5, 0x30, 0x9, 0x0, 0x6, 0x20, 0x9, 0x0,|
42 |
43 |    /* U+0024 "$" */|
```



在使用字库的地方设置字库，后面就可以使用中文：

```
/*Create a Tab view object*/
lv_obj_t *tabview;
tabview = lv_tabview_create(lv_scr_act(), LV_DIR_TOP, 50);

//设置字库
lv_obj_set_style_text_font(tabview, &lv_font_simsun_16_cjk, 0);

/*Add 3 tabs (the tabs are page (lv_page) and can be scrolled*/
lv_obj_t *tab1 = lv_tabview_add_tab(tabview, "总电池电量");
lv_obj_t *tab2 = lv_tabview_add_tab(tabview, "电池状态");
lv_obj_t *tab3 = lv_tabview_add_tab(tabview, "电池包信息");
lv_obj_t *tab4 = lv_tabview_add_tab(tabview, "历史数据");
```

七、其他

平台自带的 QT 界面会影响 LVGL 绘制屏幕的事件。如果你的 UI 界面不想被系统已有的 QT 界面和事件影响，可以在自启动的脚本文件中关闭 QT 的 ui。

打开/etc/rc.local 的文件，注释以下两行脚本即可：

```
sudo vi /etc/rc.local
```

```
linux@ubuntu: ~/lvgl/my_lvgl/lv_port_linux_frame_buffer
File Edit View Search Terminal Help
#!/bin/sh -e
#
# rc.local
#
# This script is executed at the end of each multiuser runlevel.
# Make sure that the script will "exit 0" on success or any other
# value on error.
#
# In order to enable or disable this script just change the execution
# bits.
#
# By default this script does nothing.

echo 30000 > /proc/sys/vm/min_free_kbytes
source /etc/profile
# 旧版叫QDesktop, 新版UI叫systemui
/opt/ui/systemui >/dev/null 2>&1 &
exit 0

~
~
- /etc/rc.local 11/20 55%
```

用#注释掉这两行

自启动脚本文件名

八、LVGL 的跨线程 UI 更新机制

在 LVGL 中，更新界面控件必须在主线程（即 LVGL 的任务处理线程）中进行，因为 LVGL 本身不是线程安全的。因此，如果你在另一个线程中需要更新界面，你需要通过某种线程间通信机制（如队列、邮箱、信号量等）将更新请求发送到主线程，然后在主线程中执行实际的更新操作。

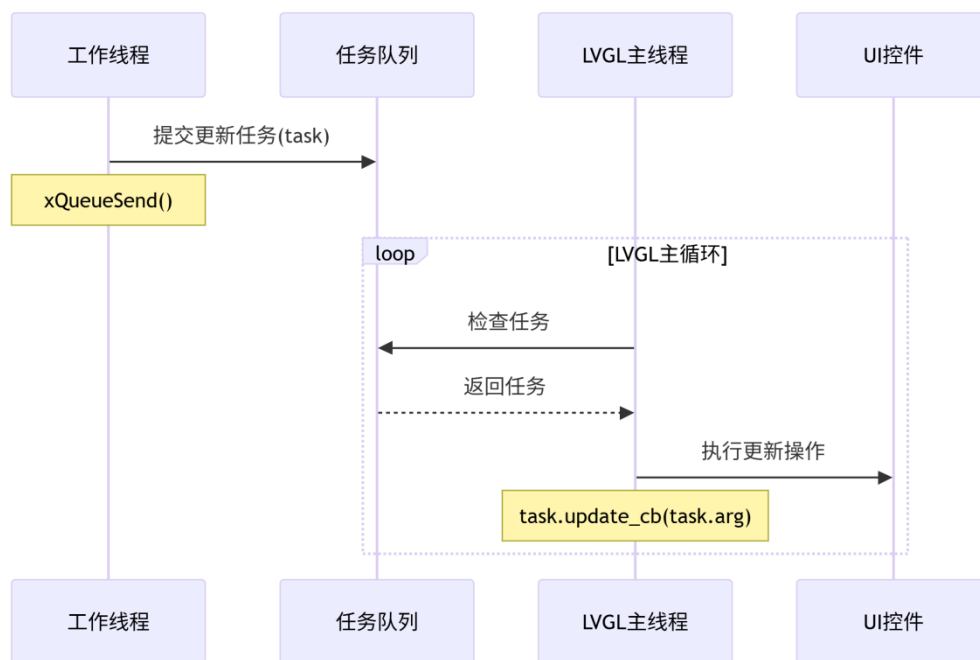
以下是实现这一目标的一般步骤：

1. **创建一个线程安全的队列**：用于将更新任务从其他线程传递到主线程。这个队列可以存储函数指针和参数，或者存储描述更新操作的结构体。

2. ****在非主线程中，将更新操作封装成一个任务****：并将任务放入队列中。

3. ****在主线程中（通常是 LVGL 的任务处理循环）****：检查队列，如果有任务，则取出并执行。

4. ****使用互斥锁保护队列****：因为多个线程可能同时访问队列。



```
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <stdlib.h>
#include <lvgl/lvgl.h>
```

```
// 定义任务队列
#define MAX_TASKS 20
```

```
typedef struct {
    void (*task_func)(void*);
    void* arg;
} lvgl_task_t;
```

```
static lvgl_task_t task_queue[MAX_TASKS];
```

```
static int task_count = 0;
static pthread_mutex_t queue_mutex = PTHREAD_MUTEX_INITIALIZER;
```

```
// UI 元素
```

```
static lv_obj_t* label;
static lv_obj_t* bar;
static lv_obj_t* switch_obj;
static lv_obj_t* slider;
```

```
// 添加任务到队列
```

```
void add_lvgl_task(void (*task_func)(void*), void* arg) {
    pthread_mutex_lock(&queue_mutex);

    if (task_count < MAX_TASKS) {
        task_queue[task_count].task_func = task_func;
        task_queue[task_count].arg = arg;
        task_count++;
    } else {
        printf("任务队列已满！ \n");
    }

    pthread_mutex_unlock(&queue_mutex);
}
```

```
// 执行队列中的所有任务
```

```
void process_lvgl_tasks() {
    pthread_mutex_lock(&queue_mutex);

    for (int i = 0; i < task_count; i++) {
        task_queue[i].task_func(task_queue[i].arg);
    }
    task_count = 0;

    pthread_mutex_unlock(&queue_mutex);
}
```

```
// 更新标签文本
```

```
void update_label_text(void* arg) {
    const char* text = (const char*)arg;
    lv_label_set_text(label, text);
}
```

```
// 更新进度条值
```

```
void update_bar_value(void* arg) {
```

```

        int value = *(int*)arg;
        lv_bar_set_value(bar, value, LV_ANIM_ON);
    }

// 切换开关状态
void toggle_switch(void* arg) {
    (void)arg; // 未使用参数
    lv_obj_toggle_state(switch_obj, LV_STATE_CHECKED);
}

// 更新滑块位置
void update_slider_value(void* arg) {
    int value = *(int*)arg;
    lv_slider_set_value(slider, value, LV_ANIM_ON);
}

// 创建 UI 界面
void create_ui() {
    lv_obj_t* scr = lv_scr_act();

    // 创建标题
    lv_obj_t* title = lv_label_create(scr);
    lv_label_set_text(title, "LVGL 多线程更新演示");
    lv_obj_align(title, LV_ALIGN_TOP_MID, 0, 20);
    lv_obj_set_style_text_font(title, &lv_font_montserrat_24, 0);

    // 创建标签
    label = lv_label_create(scr);
    lv_label_set_text(label, "初始文本");
    lv_obj_align(label, LV_ALIGN_TOP_MID, 0, 70);
    lv_obj_set_style_text_font(label, &lv_font_montserrat_18, 0);

    // 创建进度条
    bar = lv_bar_create(scr);
    lv_obj_set_size(bar, 200, 20);
    lv_obj_align(bar, LV_ALIGN_CENTER, 0, -40);
    lv_bar_set_range(bar, 0, 100);
    lv_bar_set_value(bar, 0, LV_ANIM_OFF);

    // 创建开关
    lv_obj_t* switch_label = lv_label_create(scr);
    lv_label_set_text(switch_label, "开关状态:");
    lv_obj_align(switch_label, LV_ALIGN_CENTER, -60, 20);

```

```

switch_obj = lv_switch_create(scr);
lv_obj_align(switch_obj, LV_ALIGN_CENTER, 20, 20);

// 创建滑块
slider = lv_slider_create(scr);
lv_obj_set_size(slider, 180, 20);
lv_obj_align(slider, LV_ALIGN_CENTER, 0, 60);
lv_slider_set_range(slider, 0, 100);
}

// 工作线程函数
void* worker_thread(void* arg) {
    (void)arg;
    int counter = 0;

    while (1) {
        usleep(500000); // 500ms

        // 更新标签
        char text[64];
        time_t now = time(NULL);
        struct tm* t = localtime(&now);
        snprintf(text, sizeof(text), "更新时间: %02d:%02d:%02d",
                 t->tm_hour, t->tm_min, t->tm_sec);
        add_lvgl_task(update_label_text, text);

        // 更新进度条
        int bar_value = (counter % 100);
        add_lvgl_task(update_bar_value, &bar_value);

        // 每 2 秒切换一次开关
        if (counter % 4 == 0) {
            add_lvgl_task(toggle_switch, NULL);
        }

        // 更新滑块
        int slider_value = 50 + 50 * sin(counter * 0.5);
        add_lvgl_task(update_slider_value, &slider_value);

        counter++;
    }

    return NULL;
}

```

```
int main() {
    // 初始化 LVGL
    lv_init();

    // 初始化显示和输入设备（在真实硬件上需要实现）
    // 这里使用虚拟驱动
    lv_disp_drv_t disp_drv;
    lv_disp_drv_init(&disp_drv);
    disp_drv.hor_res = 480;
    disp_drv.ver_res = 320;
    lv_disp_drv_register(&disp_drv);

    // 创建 UI
    create_ui();

    // 创建工作线程
    pthread_t thread;
    if (pthread_create(&thread, NULL, worker_thread, NULL) != 0) {
        perror("无法创建工作线程");
        return 1;
    }

    // 主循环
    while (1) {
        lv_timer_handler();
        process_lvgl_tasks();
        usleep(5000); // 5ms
    }

    return 0;
}
```